SYSTEM DESIGN

Chapter 60: Design a Payment System

Personal Notes

Payment systems have unique demands unlike anything covered so far. In social feeds, a lost message is a minor annoyance. In URL shorteners, showing stale data is fine. But in payments: charge a customer twice — you get a chargeback and a lawsuit. Charge them nothing — you lose revenue. Get the ledger wrong by one cent — regulators come knocking. The consistency requirements are absolute, and "eventual consistency" isn't tolerable.

This chapter designs a Stripe/PayPal-style payment system. It covers the FIVE hard problems: (1) IDEMPOTENCY at scale (network flaps must never cause double charges), (2) the DOUBLE-ENTRY LEDGER (the accounting foundation that makes money math bulletproof), (3) SAGAS for coordinating distributed operations across multiple services, (4) RECONCILIATION with external systems (banks, card networks), and (5) PCI DSS compliance. If you're at a fintech (Trackk, JM, or elsewhere), this is the design that underpins your production reality.

1. The Problem

Design a service that accepts payment requests from merchants, charges customers via credit cards / UPI / bank transfers, and settles funds to the merchant's account. Handle refunds, disputes, and reconciliation with banks and card networks. Never lose or duplicate money. Must be auditable end-to-end.

```
Merchant (e.g., Amazon):
   "Charge $50 to card ending 4242 for order #987"
         |
         ▼
Payment Service:
   Validate → Idempotency check → Fraud check → Send to card network
         |
         ▼
Card Network (Visa, Mastercard):
   Route to customer's issuing bank
         |
         ▼
Issuing Bank:
   Debit customer's account → Return authorization code
         |
         ▼
Payment Service:
   Update ledger → Mark transaction succeeded → Notify merchant

Later (T+2 days):
   Card network settles funds → Merchant's bank credited
```

```
        Payment service reconciles: expected == actual? Alert on mismatch.
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Merchants can charge customers (create a payment intent, then confirm/capture)
- Customers can pay via multiple methods (cards, UPI, bank transfer, wallets)
- Refunds — full or partial
- Disputes / chargebacks — customer-initiated reversal
- Payouts — settle funds to merchants on a schedule
- Ledger of all money movements — auditable, immutable
- Reconcile with external banks/networks daily

2.2 Non-Functional Requirements

- CONSISTENCY — every money movement must be exactly-once; no lost or duplicated funds
- AVAILABILITY — payment failures block business; 99.99%+ uptime required
- LATENCY — payment confirmation < 2 seconds; async parts can take minutes
- DURABILITY — every ledger entry retained forever (regulatory requirement)
- AUDITABILITY — must reconstruct exactly what happened at any point in time
- SECURITY — PCI DSS compliance; encrypt sensitive data at rest + in transit

2.3 Constraints to Confirm

- Scale — Stripe's ballpark: ~2000 TPS peak; large-scale like UPI in India: 100K+ TPS
- Which payment methods? — cards + UPI covers 90% globally
- Refund window — 30 days? 90 days? Longer for disputes
- Multi-currency? — currency conversion adds complexity

> **Consistency is non-negotiable:** Every other design in this series (feeds, chat, autocomplete) tolerated eventual consistency. Payments do NOT. Under CAP theorem, payment systems pick CONSISTENCY over availability during partitions — better to reject a payment temporarily than accept a duplicate charge. This changes fundamental design choices.

3. Step 2 — Capacity Estimation

Throughput

```
    Stripe-scale: ~2000 transactions/sec (TPS) peak
    UPI-scale (India, national network): ~100K TPS peak
    Small fintech (Trackk-scale): ~50-500 TPS
```

```
Each transaction involves:
   - Auth request to card network (~200-800 ms external)
   - Multiple ledger writes (source account, dest account, fees)
   - Fraud check (~100 ms)
   - Notification to merchant
```

Storage

```
Transactions per year: 2000 TPS × 86,400 × 365 = ~63B/year
Per record: 500 B (metadata) + 200 B (ledger entries × 2-4)

Annual raw storage: ~50 TB/year
After compression + tiered storage: ~10 TB hot, cold tiers for older

Regulatory retention: typically 7-10 years — plan for petabytes.
```

Ledger Size

```
Each transaction = 2-4 ledger entries (double-entry)

63B transactions × 3 avg entries = 190B ledger entries/year
Each entry: ~200 B
Annual: ~40 TB/year of ledger data

Ledger is APPEND-ONLY — never updated, never deleted.
Perfect for LSM-tree stores like Cassandra, or immutable log stores.
```

4. Step 3 — API Design

```
# Two-step charge (Stripe-style)

POST /v1/payment_intents
Headers: Idempotency-Key: xyz-abc-123
{
  "amount": 5000,             # in cents
  "currency": "USD",
  "customer_id": "cus_42",
  "payment_method": "pm_visa_4242",
  "metadata": { "order_id": "ord_987" }
}
→ 201 { "id": "pi_1", "status": "requires_confirmation", "client_secret":
"..." }

POST /v1/payment_intents/pi_1/confirm
→ 200 { "id": "pi_1", "status": "succeeded", "amount_captured": 5000 }

POST /v1/refunds
Headers: Idempotency-Key: xyz-abc-124
```

```
{
  "payment_intent": "pi_1",
  "amount": 2000              # partial refund
}
→ 201 { "id": "re_1", "status": "succeeded", "amount": 2000 }

GET /v1/payment_intents/pi_1
→ 200 { full state + audit history }

GET /v1/balance_transactions   # ledger view
→ 200 { list of ledger entries with running balance }
```

The Idempotency-Key header is the linchpin: Every mutating request MUST carry a client-supplied idempotency key. If the client retries after a network flap, the server sees the same key and returns the previous result WITHOUT creating a duplicate charge. This one convention prevents 99% of double-charge bugs in payment systems. Stripe pioneered this pattern; almost every serious payment API now uses it.

5. Step 4 — Data Model

```
Table: payment_intents
    id (PK) | merchant_id | customer_id | amount | currency | status
            | payment_method | idempotency_key | created_at | updated_at
    Indexes: (idempotency_key + merchant_id) UNIQUE  ← for dedup
             (customer_id, created_at)               ← for user history

Table: transactions          ← Each attempt/state change
    id (PK) | payment_intent_id | type (auth/capture/refund) | status
           | external_id (from card network) | attempt_number

Table: ledger_entries        ← APPEND ONLY. Never updated.
    id (PK) | transaction_id | account_id | direction (DR/CR) | amount
           | currency | created_at
    PARTITIONED by created_at (monthly buckets)

Table: accounts              ← Chart of accounts
    id (PK) | type (customer/merchant/fee/gateway_holding) | currency

Table: refunds
    id | payment_intent_id | amount | status | reason | created_at

Table: reconciliation_status  ← daily settlement tracking
    date | source (visa/mc/upi) | expected_amount | actual_amount | status

Sensitive data (card numbers) STORED SEPARATELY in a PCI-compliant vault
via TOKENIZATION — application only stores tokens, never raw card data.
```

6. The Five Hard Problems

Payment systems have five defining challenges. The rest of this chapter drills into each:

1. IDEMPOTENCY — network retries must not create duplicate charges
2. The DOUBLE-ENTRY LEDGER — bulletproof accounting foundation
3. DISTRIBUTED TRANSACTIONS via SAGAS — coordinating multi-service operations
4. RECONCILIATION — matching our records with external systems (banks, networks)
5. PCI COMPLIANCE — legal + security requirements around card data

7. Deep Dive: Idempotency

Client sends POST /payment_intents. Server processes it, but the response gets lost due to network timeout. Client retries. Without idempotency: server processes it AGAIN — customer is charged twice. This is unacceptable.

The Solution — Idempotency Keys

- Client generates a unique key (UUID) per LOGICAL request
- Client includes key as Idempotency-Key header
- Server stores (key, response) pair on first successful call
- Server returns cached response for any retry with the same key

```
class IdempotencyService {
    // Redis for hot lookups; Postgres for durable store
    public Response process(String key, Request req, Handler h) {
        // Try to claim the key atomically
        boolean claimed = redis.setnx("idem:" + key, "in_progress");
        redis.expire("idem:" + key, 24 * 3600);

        if (!claimed) {
            // Someone else is processing OR already done
            String cached = redis.get("idem:" + key);
            if (cached.equals("in_progress")) {
                return Response.wait();      // client should poll
            } else {
                return Response.fromJson(cached);   // return cached
result
            }
        }

        // We own this key — process the request
        try {
            Response resp = h.handle(req);
            redis.set("idem:" + key, resp.toJson());
            db.insertIdempotencyRecord(key, resp);   // durable
            return resp;
```

```
        } catch (Exception e) {
            redis.del("idem:" + key);   // failure — allow retry
            throw e;
        }
    }
}
```

Idempotency has a subtle correctness rule: The idempotency key is UNIQUE PER LOGICAL OPERATION. If a client sends POST /refunds twice with the SAME key, both must produce the same refund. But if a client sends refund #1 with key X and refund #2 (different logical refund) with key X, that's a client bug — server should reject. Stripe returns 409 Conflict if the key was used for a different request body.

Retention

Idempotency keys are stored for 24-72 hours typically. After that, retries fail — but at that point, retries indicate a serious client-side bug, not a network glitch. Stripe uses 24 hours; internal APIs sometimes 7 days.

8. Deep Dive: The Double-Entry Ledger

Money doesn't appear or disappear — it MOVES. Every payment involves at least two accounts: money leaves one, arrives at another. DOUBLE-ENTRY BOOKKEEPING (invented in 1494!) captures this: every transaction has TWO entries, DEBIT one account, CREDIT another. Amounts must balance.

The Rules

- Every transaction produces AT LEAST 2 ledger entries
- Sum of DEBITS = Sum of CREDITS (must always balance)
- Ledger entries are IMMUTABLE — never update, never delete
- Corrections are made via REVERSING ENTRIES (new entries that cancel old ones)
- Account balance = sum of all its historical entries

Example — Simple $50 Payment

```
Alice pays $50 to Bob's merchant account.
Payment platform takes a 3% fee = $1.50. Bob receives $48.50.

Ledger entries for this transaction:
```

Entry	Account	Direction	Amount (USD)
E1	Alice's card	DEBIT	50.00
E2	Bob's balance	CREDIT	48.50
E3	Platform fees	CREDIT	1.50

```
Verify: Debits = 50.00
        Credits = 48.50 + 1.50 = 50.00
        BALANCED ✓

Alice's card account: reduced by $50 (debit)
Bob's balance:        increased by $48.50 (credit)
Platform fees:        increased by $1.50 (credit)
```

Why Double-Entry Is Bulletproof

- If you accidentally credit an account, the imbalance is immediately visible — no silent corruption
- Every entry references its source transaction — trivially auditable
- Balance can be recomputed from history at any point in time
- Reversing errors is a normal operation — no destructive edits

> **Never update balances directly:** The naive design has a `balances` table where you UPDATE the balance directly on each payment. This makes bugs SILENT — a subtracted amount that was supposed to be added gives no error, just wrong data. Real payment systems store balance as a DERIVED VIEW of the immutable ledger; you can always rebuild it. It's much safer.

9. Deep Dive: The Payment Flow (State Machine)

A payment isn't atomic — it moves through states. Explicit state machine + event log makes debugging + auditing possible.

```
REQUIRES_CONFIRMATION
   │
   ▼ merchant/customer confirms
REQUIRES_ACTION       (e.g., 3D Secure challenge)
   │
   ▼ customer completes 3DS
PROCESSING          (calling card network)
   │
   ├──▶ succeeded → SUCCEEDED
   │
   └──▶ declined → REQUIRES_PAYMENT_METHOD → (retry with new card)

From SUCCEEDED, other transitions:
   REFUNDED / PARTIALLY_REFUNDED
   DISPUTED (chargeback filed)
   FRAUDULENT (marked by fraud team)
```

State Transition Rules

- Only certain transitions are legal (can't go from CREATED to REFUNDED directly)
- Every transition emits an event to an APPEND-ONLY event log (payment_events table)
- Events include: timestamp, actor, previous state, new state, reason

- Current state is DERIVED from the event log — the log is the source of truth

```
Event log table:
    event_id | payment_id | timestamp | event_type | payload | actor

Sample events for one payment:
    event_id | timestamp        | event_type          | actor
    ─────────┼──────────────────┼─────────────────────┼──────────
    e_1      | 10:00:00.123     | CREATED             | merchant
    e_2      | 10:00:00.145     | CONFIRMED           | customer
    e_3      | 10:00:01.220     | 3DS_CHALLENGED      | system
    e_4      | 10:00:15.010     | 3DS_COMPLETED       | customer
    e_5      | 10:00:15.150     | AUTHORIZATION_SENT  | system
    e_6      | 10:00:16.800     | AUTHORIZED          | card_network
    e_7      | 10:00:16.900     | CAPTURED            | system
    e_8      | 10:00:17.100     | SUCCEEDED           | system

     REBUILD state by replaying events in order.
     Called EVENT SOURCING — the source of truth is the event log.
```

10. Deep Dive: Sagas for Distributed Transactions

A single payment touches: card network, fraud service, ledger service, merchant notification, analytics pipeline. Each is a separate service. If we could wrap them all in one database transaction, life would be easy. But you can't ACID-transact across services. Enter the SAGA pattern.

The Idea

- Break the operation into a sequence of local transactions, each in one service
- If any step fails, execute COMPENSATING actions to undo previous steps
- Each step is atomic within its own service — no global transaction needed

Example — Payment Saga

```
     Steps (forward path):

        1. reserve_funds     (customer's account) →  IF FAILS: nothing to
undo
        2. debit_customer    (ledger)             →  compensate:
credit_customer
        3. call_card_network (external)            →  compensate:
reverse_auth
        4. credit_merchant   (ledger)             →  compensate:
debit_merchant
        5. notify_merchant   (async)               →  compensate:
send_reversal

     If step 3 (card network) fails:
        Execute compensations in REVERSE order:
```

```
        - undo step 2 (credit_customer to reverse the debit)
        - undo step 1 (release reserved funds)
        Return failure to caller.

     If step 4 succeeds but step 5 fails (network flap):
        Log the failure but DON'T reverse the payment.
        Retry step 5 with exponential backoff — merchant will eventually
get notified.
        This is a "non-critical" step — doesn't warrant undoing money
movement.
```

Types of Sagas

Type	How It Works	Best For
Choreography	Each service publishes/subscribes to events; no central coordinator	Simpler for few steps; hard to reason about with many
Orchestration	A central orchestrator invokes each service in sequence	Easier to trace + debug; single point of coordination

> **Compensations are NOT rollbacks:** You can't "undo" calling a card network — the auth already happened. What you can do is call the REVERSE_AUTH endpoint to make it a no-op. Compensation is an APPLICATION-LEVEL inverse operation, not a database rollback. Every step in a saga must have a well-defined compensating action, and both must be idempotent (in case they're retried).

11. Deep Dive: Reconciliation

Our system records: "Alice paid Bob $50 at 10:00 AM." But actual money movement happens through card networks and banks — separate systems. Do THEIR records match OURS? Reconciliation is the daily/hourly process of comparing.

Two Kinds of Reconciliation

- INTERNAL — Our ledger vs our various services (payment DB vs event log vs analytics). Should always match exactly.
- EXTERNAL — Our records vs bank/card network records. Discrepancies are common but must be investigated.

The Process

6. Daily settlement file arrives from card network (typically SFTP dropped CSV)
7. Load into staging table
8. Match records: our_txn_id == external_reference
9. Bucket into: (a) matched, (b) our-side only, (c) their-side only, (d) amount mismatch
10. Automatically resolve simple cases (e.g., timing differences)

11. Alert humans for genuine discrepancies — money at risk

```
     Buckets after daily match:

     MATCHED         (99.9% ideal)
        Our records and their records agree exactly. Done.

     OUR SIDE ONLY   (~0.05% typical)
        We recorded a transaction; they didn't process it.
        Possible causes: network failure after we sent auth, timing.
        Action: query their system; if truly not processed, reverse our
ledger entry.

     THEIR SIDE ONLY (~0.05%)
        They processed a transaction we don't know about.
        Very serious — potential double-processing or bug.
        Action: locate our transaction; if genuinely missing, back-fill or
investigate.

     AMOUNT MISMATCH (~0.01%)
        Same transaction id, different amount.
        Could be partial capture, currency conversion, fee difference.
        Action: reconcile manually; adjust ledger with reversing entry if
needed.
```

Reconciliation is where you catch bugs: Most payment bugs first surface during reconciliation. That's why it's a critical daily process — not just compliance theater. If reconciliation drifts 0.1% one day, then 0.5% the next, you know something's wrong before regulators or customers notice. Fintech teams monitor reconciliation dashboards obsessively.

12. Deep Dive: PCI DSS Compliance

PCI DSS (Payment Card Industry Data Security Standard) governs how card data is handled. Non-compliance = fines + losing ability to process cards. Every serious payment system MUST comply.

The Big Rules

- Full card numbers (PAN) NEVER stored in your database
- If you accept cards, use TOKENIZATION — store an opaque token that a compliant vault can dereference
- If you MUST handle raw card data, isolate that service (Cardholder Data Environment) with strong access controls
- All data encrypted in transit (TLS 1.2+) and at rest (AES-256)
- Access to card data is audited — who accessed what, when
- Regular vulnerability scans + annual penetration tests

Tokenization Flow

```
1. Customer enters card 4242-4242-4242-4242 in merchant site.
2. Card data submitted DIRECTLY to Payment Gateway (Stripe/etc.),
   bypassing merchant's servers entirely (via JavaScript SDK).
3. Gateway returns a TOKEN: "pm_1A2B3C4D".
4. Merchant stores the token in its DB.
5. To charge: merchant sends {token, amount} to Gateway;
   Gateway looks up token → real card → processes.

Merchant NEVER sees or stores the real card number.
PCI scope is dramatically reduced.
```

> **PCI has 4 compliance levels:** Level 1 (largest, 6M+ transactions/year) requires annual on-site audit by QSA. Level 4 (smallest) requires self-assessment. If you handle cards at scale, you WILL be Level 1 — plan for the ongoing operational overhead. Most companies outsource card handling to Stripe/Braintree/Adyen precisely to shrink their PCI scope.

13. Deep Dive: Fraud Detection Integration

Fraud detection is usually a separate SERVICE that the payment system calls synchronously before authorizing. The design decision: how tightly to couple them.

Placement

- BEFORE card network call — block fraudulent auths, save network fees
- SYNCHRONOUS in the hot path — adds ~50-200 ms but is necessary
- Failure mode: fraud service down → decide policy (fail-open vs fail-closed)

Signals to Send

- Transaction amount, merchant, currency
- Customer history (past transactions, chargebacks, account age)
- Device fingerprint, IP address, geolocation
- Velocity signals (how many transactions in last hour from this IP/card)
- Payment method details (card country vs shipping country mismatch)

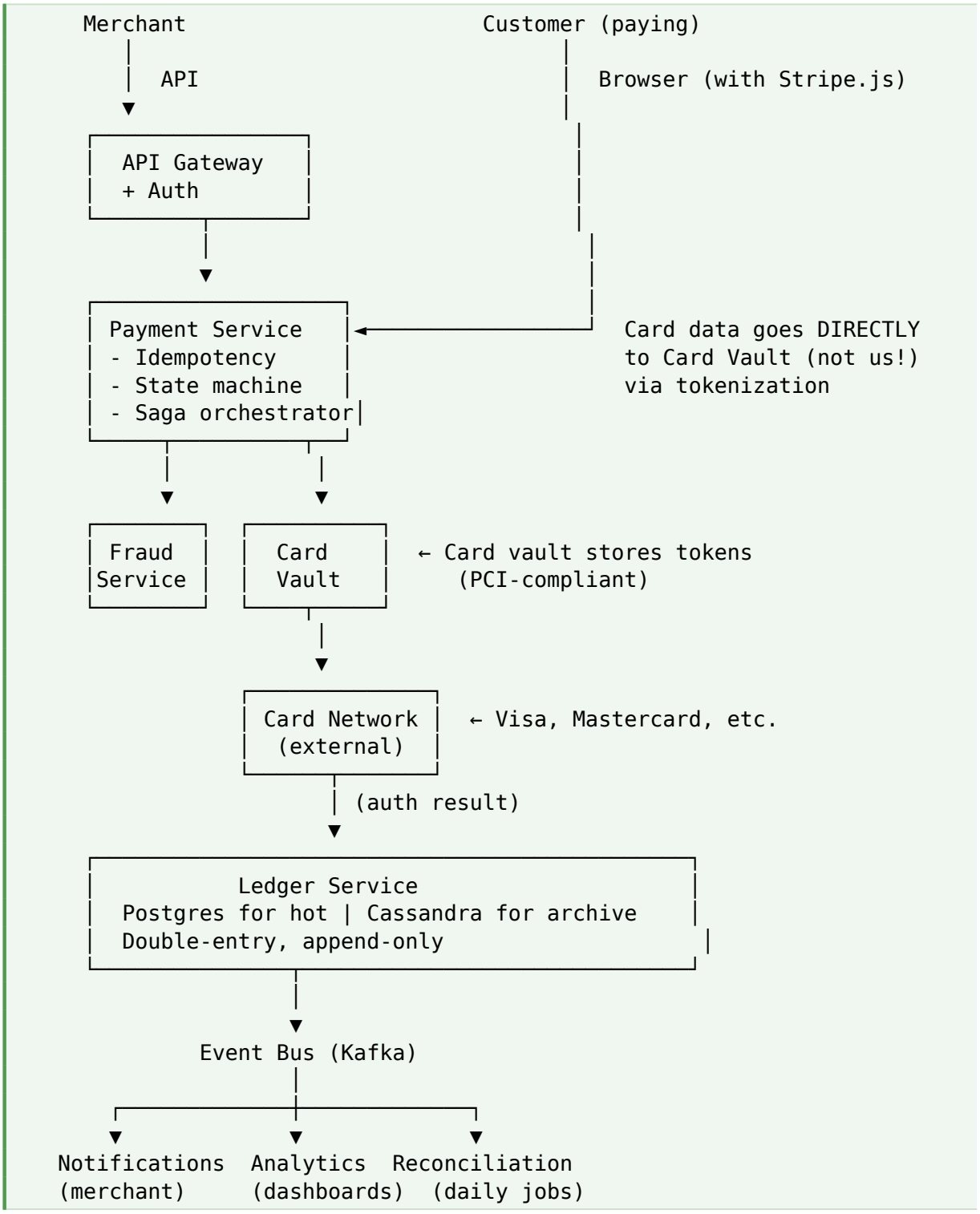
Response

- APPROVE — score below threshold, proceed to card network
- REVIEW — score suspicious; hold for manual review or 3D Secure challenge
- DECLINE — score too high, block outright

> **3D Secure is your friend:** When fraud risk is medium, don't block outright — challenge with 3D Secure (customer confirms with their bank via SMS/app). If they complete it, liability for chargeback

shifts to the issuing bank. Higher friction but lower fraud losses. Balance is a tunable business decision.

14. Step 5 — High-Level Architecture



15. Additional Features

15.1 Payouts

Merchants don't get paid instantly — funds settle T+1 or T+2 days. Batch payout job: for each merchant, sum all successful transactions in the window, subtract fees, initiate ACH/wire transfer. Track payouts as their own entities with their own ledger entries.

15.2 Multi-Currency

Customer pays in EUR, merchant wants USD. Payment system: converts at spot rate (usually with a 1-2% margin). Record BOTH currencies in ledger — one entry per side. Currency conversion adds a THIRD ledger entry to capture the margin income.

15.3 Subscriptions / Recurring Billing

Store payment method (tokenized card). On billing anniversary, schedule a charge. If it fails, retry with exponential backoff (day 1, day 3, day 7). After N failures, cancel subscription. Every retry is idempotent using a deterministic idempotency key (subscription_id + billing_period).

15.4 Disputes / Chargebacks

Customer disputes with their bank; bank pulls the money back from us (retroactive chargeback). Payment system: record dispute event, freeze merchant funds if needed, gather evidence, submit to card network. Dispute lifecycle can span 60+ days. Chargebacks are a major fraud loss vector — automate evidence gathering aggressively.

16. Failure Modes

Failure	Mitigation
Card network timeout	Do NOT retry blindly — you don't know if the auth went through. Poll for status, or reconcile later.
Idempotency service down	REJECT new payments (fail-closed) — never process without idempotency guarantee.
Ledger service down	Reject payments — cannot record without ledger. Merchants queue retries.
Fraud service down	Fail-CLOSED for high-risk cases; fail-open for known-good customers. Business decision.
Duplicate merchant webhook	Merchant retries webhook delivery. Fine — merchants use idempotency keys on their end.
Reconciliation mismatch	Alert immediately. Freeze payouts until resolved. Manual investigation.

Failure	Mitigation
PCI vault down	Cannot process new cards. Existing tokenized cards may still work if we cached them briefly.

> **Payment systems fail CLOSED:** This is the opposite of most systems (e.g., autocomplete fails open in Ch 59). If any critical component is down, REJECT the payment — never process without full guarantees. The cost of a false negative (declined valid payment) is much lower than a false positive (accepted invalid payment with lost funds). This is the FINTECH mindset.

17. Trade-offs Summary

Decision	Trade-off
Sync vs async saga	Sync = simpler, blocks thread; async = fast but requires message bus + retries
Orchestration vs choreography	Orchestration = clear flow; choreography = decoupled but harder to trace
Tokenization vs direct card handling	Tokenization = 90% less PCI scope; direct = required if you're the vault
Fail-open vs fail-closed fraud	Open = uptime, revenue risk; closed = safe, lost sales
Real-time vs batch reconciliation	Real-time = catch issues instantly; batch = simpler + adequate for most
Event sourcing yes/no	Yes = complete audit + replay; higher storage + rebuild complexity
Single vs multi-region	Single = simpler; multi = HA + regulatory (data residency)
Retry aggressively vs conservatively	Aggressive = better UX; risk of duplicates if idempotency imperfect

18. Common Mistakes

Mistake 1: No idempotency

Payment API without an Idempotency-Key header is unfit for production. Every mutating endpoint MUST accept + honor idempotency keys. Retries WILL happen — network flaps, timeouts, client bugs. Without idempotency, they cause double charges. This is the #1 payment system bug.

Mistake 2: Updating balances directly

Storing balance as a mutable column that gets UPDATE'd is a recipe for silent corruption. Use double-entry ledger; balance is DERIVED from immutable entries. Bugs become visible immediately (imbalance) instead of silent.

Mistake 3: Trying to use ACID across services

You can't wrap a call to Stripe + your DB + your notification service in one transaction. That's what sagas are for. Explain the saga pattern with compensations, even if the interviewer expects you to know.

Mistake 4: Not planning for reconciliation

A payment system that can't reconcile with external banks is broken by design. Discuss the daily match process, discrepancy buckets, and alerting from day one.

Mistake 5: Storing raw card numbers

PCI DSS non-compliance = massive fines + loss of card processing ability. Tokenize via a PCI-compliant vault. Never store PAN in your app database — even encrypted.

Mistake 6: Retrying card network calls blindly

Card network timed out? DO NOT retry — you don't know if the auth succeeded on their side. Instead: mark transaction as UNKNOWN, poll for status, or wait for reconciliation. Blind retries are the #2 cause of double charges.

Mistake 7: Fail-open on critical services

If idempotency service is down, DON'T just process payments without checks. Fail-closed: reject new payments until the service recovers. In finance, safety > availability.

19. Best Practices

- Idempotency-Key HEADER on every mutating endpoint — non-negotiable
- Double-entry LEDGER with immutable, append-only entries — never update balances directly
- EVENT SOURCING for payment state — reconstruct state from event log
- SAGAS for cross-service operations with well-defined compensations
- Daily RECONCILIATION with external systems + alerting on discrepancies
- TOKENIZATION for card data — massively reduces PCI scope
- FAIL-CLOSED for critical dependencies (fraud, ledger, idempotency)
- Never retry card network calls blindly — poll or reconcile instead
- 3D Secure to shift chargeback liability for medium-risk transactions
- Encrypted at rest + in transit; audit trails for every access to sensitive data

20. Quick Reference

Concept	Meaning
Idempotency key	Client-supplied UUID; server dedupes retries by it
Double-entry ledger	Every txn: debits = credits; append-only, immutable
Debit / Credit	Direction of a ledger entry (money in vs out of an account)
Event sourcing	State derived from event log — the log is source of truth
Saga	Long-running distributed transaction with compensating steps
Compensating action	Undoes a previous step (application-level, not DB rollback)
Reconciliation	Daily match of our ledger vs external bank/network records
Tokenization	Replace card number with opaque token; real data in a vault
PCI DSS	Card industry security standard; 4 compliance levels
3D Secure	Customer confirms with bank; shifts liability if fraud
Fail-closed	When critical service down, REJECT — never process unsafely
Chargeback	Customer's bank pulls money back — retroactive reversal
Settlement (T+1/T+2)	Funds actually move 1-2 days after authorization

21. Related Interview Problems

Payment system patterns apply broadly to any system where correctness matters more than speed:

12. Design a Wallet / Digital Money System (Paytm, Venmo, PayPal balance)
13. Design a Stock Trading Platform — order matching, positions, fills, settlements
14. Design a Banking Core — accounts, deposits, withdrawals, transfers
15. Design a Crypto Exchange — similar to trading but with on-chain settlement
16. Design a Loyalty Points System — small-scale ledger; same double-entry pattern
17. Design an Advertising Billing System — CPM/CPC billing at massive scale
18. Design a Subscription Management System — recurring billing, upgrades, credits
19. Design a Marketplace Payment Splitter (Stripe Connect) — split fees between multiple parties

22. Final Takeaway

Payment systems are the domain where distributed systems concepts stop being abstractions and become do-or-die requirements. Idempotency isn't a nice-to-have — without it, you get double-charged customers. Double-entry ledgers aren't paranoia — they're how you sleep at night knowing the books balance. Sagas aren't fancy — they're how you coordinate charges across five services without ACID. Reconciliation isn't compliance theater — it's how you catch bugs before regulators do.

For interviews (especially fintech ones — Stripe, PayPal, Square, or Indian fintechs like Razorpay, PhonePe, Trackk itself), memorize these five: (1) idempotency keys are mandatory, (2) double-entry ledger with immutable append-only entries, (3) sagas with compensations for cross-service work, (4) daily reconciliation with external systems, (5) tokenization + PCI compliance for card data. Then be ready for the deep dives: "What if the network times out mid-payment? How do you handle a mismatched reconciliation? What if fraud service is down?"

> **Key Takeaway:** Payment systems = CORRECTNESS > everything. IDEMPOTENCY-KEY on every mutation prevents double charges from retries. DOUBLE-ENTRY LEDGER with debits = credits, append-only, immutable — bulletproof accounting. EVENT SOURCING for payment state — replay from event log. SAGAS with compensating actions for cross-service operations (no distributed ACID). DAILY RECONCILIATION with banks/networks; alert on discrepancies. TOKENIZATION for card data — reduces PCI scope. FAIL-CLOSED for critical services (idempotency, ledger, fraud). Never retry card network calls blindly — poll or reconcile. 3D Secure shifts liability. Settlement is T+1/T+2, not instant.

23. What's Next?

Payment system done. Remaining designs and deep dives:

- Design a Notification System — cross-channel delivery, priorities, batching, dedup
- Design a Metrics/Monitoring System (Datadog-style) — time-series data, retention tiers
- Design a Distributed Job Scheduler — Cron at scale, resilient task queues
- Design a Stock Trading Platform — order matching engine, positions, market data feeds
- Design a Search Engine — the full stack integrating crawler + autocomplete
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once
- Deep Dive: Consensus Algorithms — Raft (worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing (extend Ch 60)